Computer Organization LAB

计算机 2405 邵凯伟

2026 年 11 月 5 日

目录

0 环境搭建 **3**
- 0.1 logisim . . . 3
- 0.2 verilog . . . 3

1 实验一 组合逻辑电路设计 **3**
- 1.1 logisim 实现 . . . 4
  - 1.1.1 多路复用器 . . . 4
  - 1.1.2 编码器 . . . 4
  - 1.1.3 译码器 . . . 6
- 1.2 verilog 实现 . . . 6
  - 1.2.1 多路复用器 . . . 6
  - 1.2.2 优先编码器 . . . 8
  - 1.2.3 译码器 . . . 8
- 1.3 verilog 仿真 . . . 9
  - 1.3.1 优先编码器 . . . 9
  - 1.3.2 译码器 . . . 10

2 实验二时序逻辑电路设计 **11**
- 2.1 logisim 实现 . . . 11
  - 2.1.1 S-R 锁存器 . . . 11
  - 2.1.2 D-锁存器 . . . 12
  - 2.1.3 主从 D 触发器 . . . 12
  - 2.1.4 移位寄存器 . . . 13

2.1.5 44 进制计数器 14
2.1.6 n 分频计数器 14
2.1.7 01 序列检测哭 14
2.2 verilog 实现 15
2.2.1 D 锁存器 15
2.2.2 D-触发器 15
2.2.3 移位寄存器 16
2.2.4 44 进制计数器 17
2.3 verilog 仿真 18
2.3.1 D-锁存器 18
2.3.2 D-触发器 20
2.3.3 移位寄存器 22
2.3.4 44 进制计数器 24

3 3 **26**

0 环境搭建

0.1 logisim

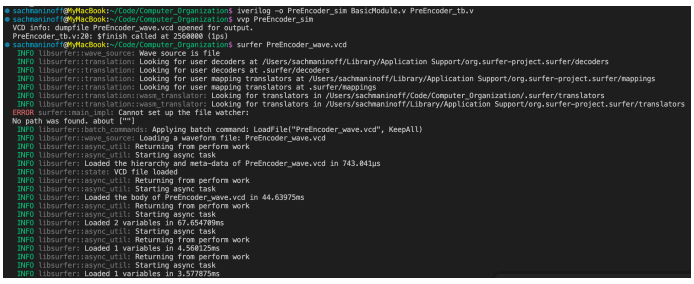
直接下载

0.2 verilog

采用 iverilog + surfer 方案 
使用`iverilog -o output code1.v code2.v`编译
使用`vvp output`模拟运行，（Verilog Virtual Processor）
使用`surfer filename.vcd`查看波形
surfer 原料: 在仿真文件中加入

```
initial begin
    $dumpfile("filenae.vcd");
    $dumpfile(0, modulename);
end
```

生成 vcd 文件
输出示例:



1 实验一 组合逻辑电路设计

基础门电路设计：多输入门电路, 复用器
基础功能模块设计：编码器, 译码器

1.1 logisim 实现

1.1.1 多路复用器

二路复用器

如图 1。

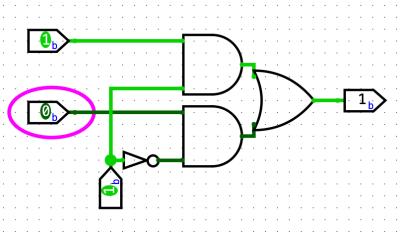


图 1: 二路复用器

三态门实现二路复用器

如图 2，用三态门代替与门，复用三态门线与省略或门。

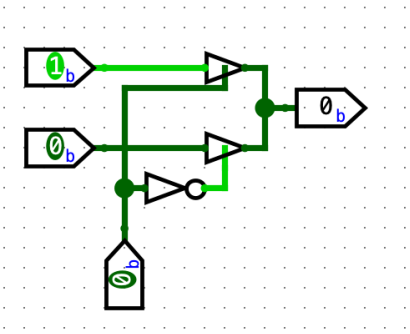


图 2: 三态门二路复用器

四路复用器

采用两层二路复用器实现，如图 3。

1.1.2 编码器

8-3 编码器

根据真值表，只需进行或逻辑:

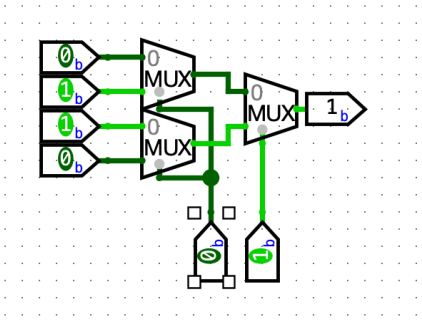


图 3: 四路复用器

$$\begin{cases} Y_2 = I_4 + I_5 + I_6 + I_7 \\ Y_1 = I_2 + I_3 + I_6 + I_7 \\ Y_0 = I_1 + I_3 + I_5 + I_7 \end{cases}$$

实现如图 4。

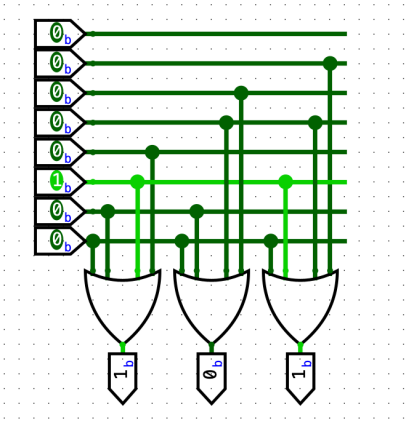


图 4: 8-3 编码器

8-3 优先编码器

逻辑分为两部分：仅点亮最高优先级的数据线，普通 8-3 编码器。

优先级部分：每一个输入 D_k 取决于 D_0 D_{k-1} 的非的与，再与上 I_k（图 5中的七个与门），非之与部分可变为或非门，（图中两个或非门用于简化低位逻辑），避免使用过多输入的与门。最后三个或门为普通 8-3 编码器。

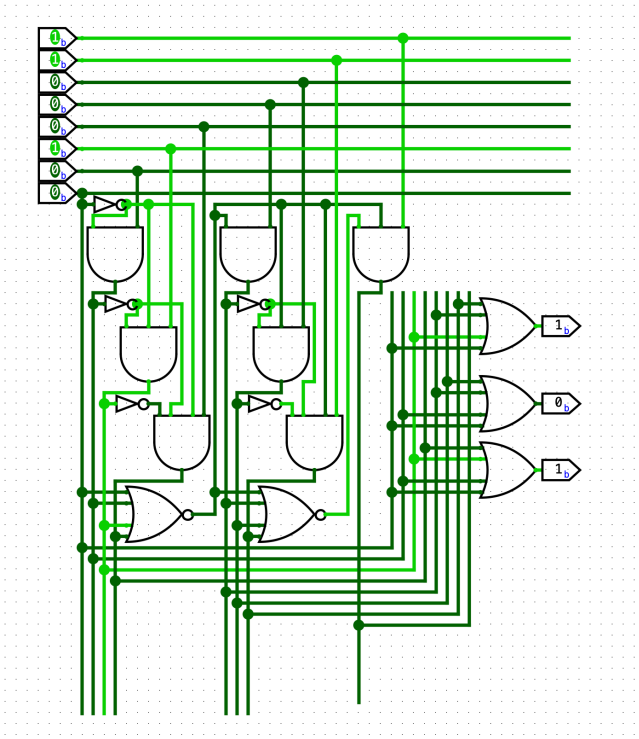


图 5: 8-3 优先编码器

1.1.3 译码器

3-8 译码器

如图 6, 使用与阵列即可。

1.2 verilog 实现

1.2.1 多路复用器

使用参数化路数量`#(parameter ways = 4)`，易于复用代码
`$clog2()`为内置函数，选择位宽为输入位宽的对数
最后将`data_in[sel]`作为输出即可。

```
module MUX #(
    parameter ways = 4
) (
    input wire [ways - 1 : 0] data_in,
```

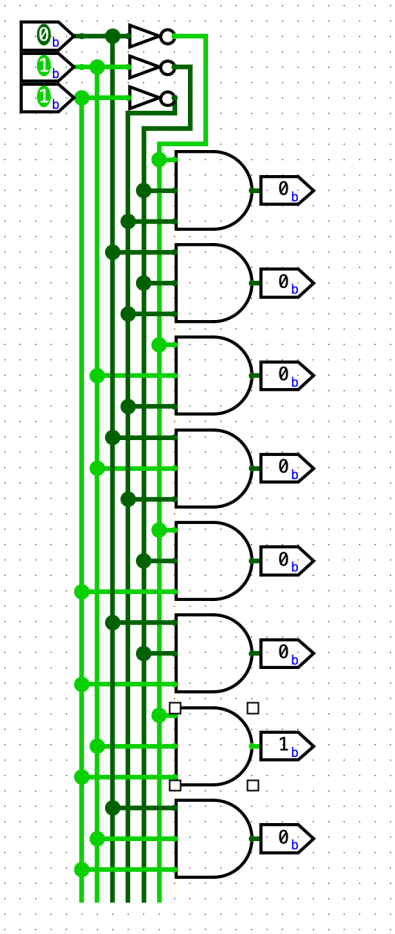


图 6: 3-8 译码器

```
    input wire [$clog2(ways) - 1 : 0] sel,
    output wire data_out
);
    assign data_out = data_in[sel];
endmodule
```

Listing 1: 多路复用器

1.2.2 优先编码器

输出在always块中赋值，将wire改为reg，用flag模拟break语句。
在for循环外先给data_out赋初值，以防止data_in全为 0 时data_out变为寄存器.

```
module PreEncoder #(
    parameter ways = 8
) (
    input wire [ways - 1 : 0] data_in,
    output reg [$clog2(ways) - 1 : 0] data_out
);
    reg flag;
    integer i;
    always @(*) begin
        flag = 0;
        i = 0;
        data_out = 0;
        for (i = ways - 1; i >= 0; i = i - 1)  begin
            if (!flag) begin
                if (data_in[i]) begin
                    flag = 1;
                    data_out = i;
                end
            end
        end
    end
endmodule
```

Listing 2: 优先编码器

1.2.3 译码器

data_out的第data_in位为 1，其余均为 0。
故data_out = 1 << data_in。

```
module Decoder #(
    parameter ways = 8
) (
    input wire [$clog2(ways) - 1 : 0] data_in,
```

```
    output wire [ways - 1 : 0] data_out
);
    assign data_out = 1'b1 << data_in;
endmodule
```

Listing 3: 译码器

1.3 verilog 仿真

1.3.1 优先编码器

```
`timescale 1ns / 1ps
module PreEncoder_tb;
    parameter ways = 8;
    reg [ways - 1 : 0] data_in;
    wire [$clog2(ways) - 1 : 0] data_out;

    integer i;

    PreEncoder #(
        .ways(ways)
    ) PreEncoder_sim (
        .data_in(data_in),
        .data_out(data_out)
    );
    initial begin
        for (i = 0; i < (1 << ways); i = i + 1) begin
            data_in = i;
            #10;
        end
        $finish;
    end
    initial begin
        $dumpfile("PreEncoder_wave.vcd");
        $dumpvars(0, PreEncoder_tb);
    end
endmodule
```

Listing 4: 优先编码器仿真

波形图

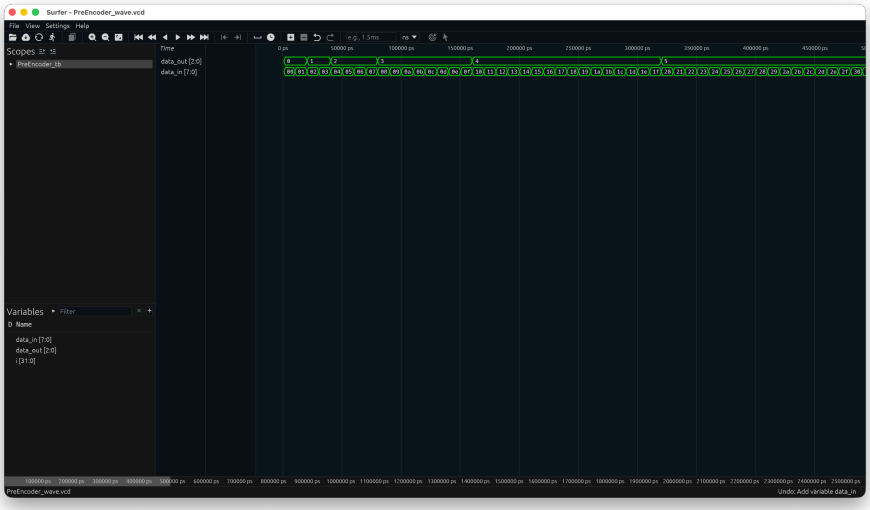


图 7: 优先编码器仿真波形

1.3.2 译码器

```
`timescale 1ns / 1ps
module Decoder_tb;
    parameter ways = 8;
    reg [$clog2(ways) - 1 : 0] data_in;
    wire [ways - 1 : 0] data_out;

    integer i;

    Decoder #(
        .ways(ways)
    ) Decoder_sim (
        .data_in(data_in),
        .data_out(data_out)
    );
    initial begin
        for (i = 0; i < ways; i = i + 1) begin
            data_in = i;
```

```
18              #10;
19          end
20          $finish;
21      end
22      initial begin
23          $dumpfile("Decoder_wave.vcd");
24          $dumpvars(0, Decoder_tb);
25      end
26  endmodule
```

Listing 5: 编码器仿真

波形图

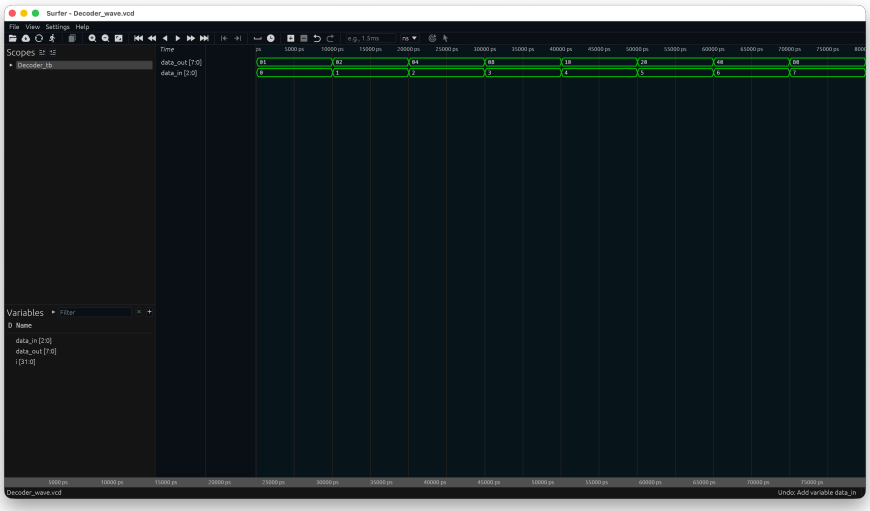


图 8: 译码器仿真波形

2 实验二时序逻辑电路设计

2.1 logisim 实现

2.1.1 S-R 锁存器

双稳态 + 或门, 如图 9

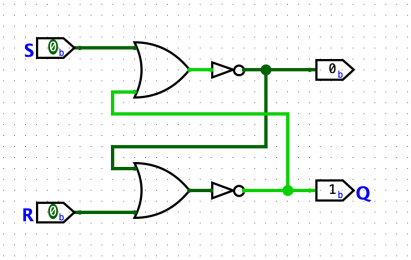


图 9: S-R 锁存器

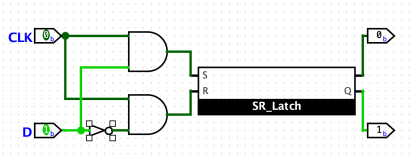


图 10: D-锁存器

2.1.2 D-锁存器

D 和/D 作为 SR 锁存器的输入, 与上 CLK 即可, 如图 10

2.1.3 主从 D 触发器

两个 D 触发器, /CLK 控制前一个, CLK 控制后一个, CLK 上沿时锁住前一个, 后一个存储当时输入

使能信号 EN: 加入一个复用器, 当 EN 为假时选择输入作为输入, EN 为真时将 D 作为输入.

复位信号 RST: 异步复位, 优先级最高, RST 为真时需要将两个锁存器的输入置 0, 即图中两个与门, 再将两个锁存器的 clk 均置为真, 即图中的两个或门, 于是就可以实现异步复位. 如图 11

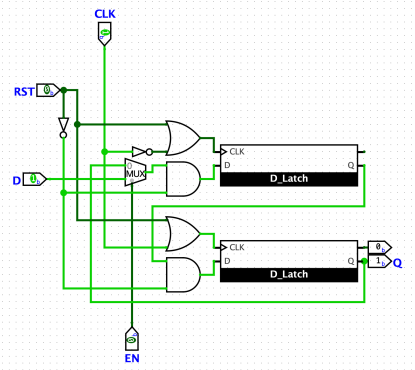


图 11: D-触发器

2.1.4 移位寄存器

以 8 位为例, 由 8 个 D 触发器组成, EN 全部接 1, RST 端和 CLK 端接输入 RST 和 CLK, 数据端由四个来源复选: 输入端, 输出端 (保持), 下一 D 触发器输出 (右移) 和上一 D 触发器输出 (左移), 由 S 控制 MUX. 最高输入端由 Algor 复选, 从 0 和自身输出二选一, 实现逻辑右移和算术右移. 最高位位的利用器和最低位的 0 输入可改为串行输入或者循环移位输入. 如图 12.

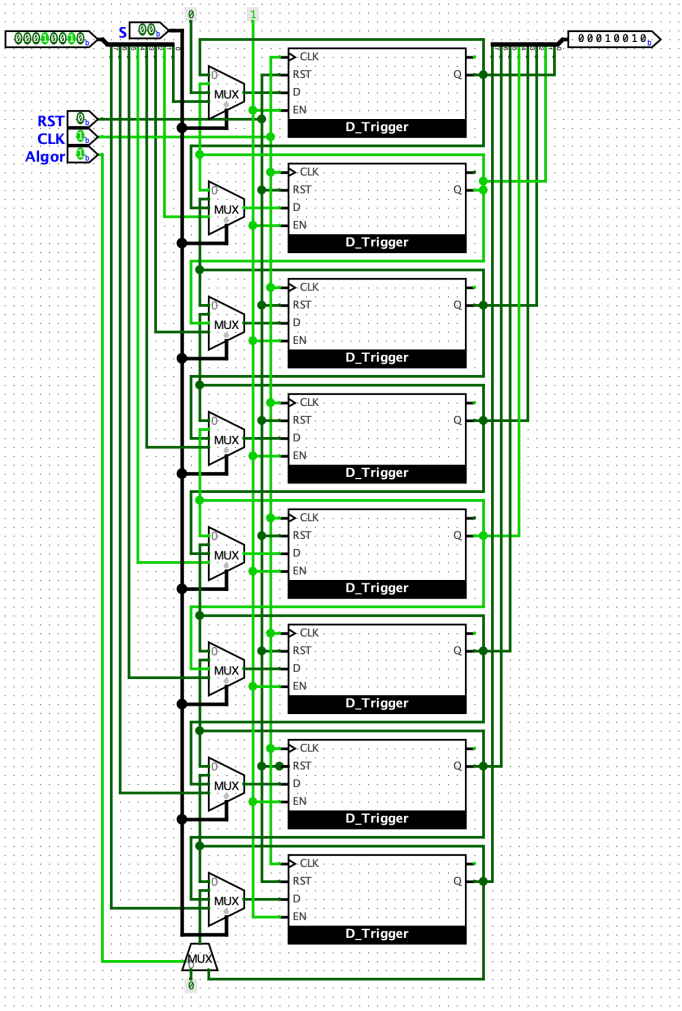


图 12: 8 位移位寄存器

2.1.5 44 进制计数器

采用强置位方案. 使用上一个实现的 8 位寄存器, 将输出与 44(0x2c) 比较, 如果相等则使能 RST 清零 (RST 实现为异步清零), 否则将输出送至加法器加 1 后作为下一周期的输入. 为实现异步清零, 将 RST 与强置位信号进行或操作后再接入寄存器的 RST. 寄存器工作模式设为 0b11(3) 并行输入. 如图 13

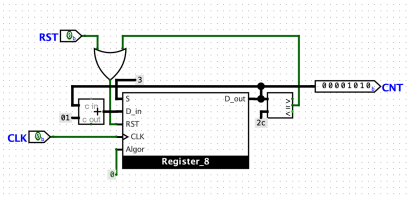


图 13: 44 进制计数器

2.1.6 n 分频计数器

将上一个计数器的输出进行与非逻辑, 再设置 n 为比较器另一端的常量即可, 实现略.

2.1.7 01 序列检测哭

时序逻辑设计如图 14, 实现如图 15

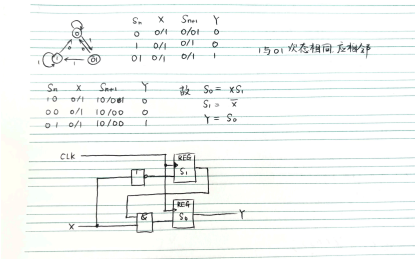


图 14: 时序逻辑设计

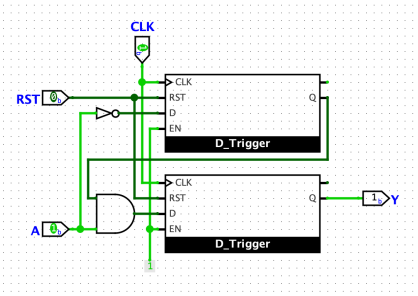


图 15: 01 序列检测器

2.2 verilog 实现

2.2.1 D 锁存器

先判断是否是复位信号, 将 Q 赋值为 0, 否则将 D 送入 Q, 注意时序逻辑应当使用 <= 赋值.

```
module D_Latch (
    input CLK,
    input D,
    output reg Q, NQ,
    input RST, EN
);
    always @(*) begin
        if (RST) begin
            Q <= 0;
            NQ <= 1;
        end
        else if (EN && CLK) begin
            Q <= D;
            NQ <= ~D;
        end
    end
endmodule
```

Listing 6: D-锁存器

2.2.2 D-触发器

基本同 D-锁存器, 只需将电平控制的 CLK(@(*)) 变为上升沿触发 (@(posedge CLK)). 可加入异步的复位与置位信号 RST 与 SET. 在块内部先判断是否是 RST 和 SET 引起, 实现高优先级.

```
module D_Trigger (
    input CLK,
    output reg Q, NQ,
    input D,
    input RST, SET, EN
);
    always @(posedge CLK or posedge RST or posedge SET) begin
```

```
        if (RST) begin
            Q <= 0;
            NQ <= 1;
        end
        else if (SET) begin
            Q <= 1;
            NQ <= 0;
        end
        else if (EN) begin
            Q <= D;
            NQ <= ~D;
        end
    end
endmodule
```

Listing 7: D-触发器

2.2.3 移位寄存器

12-14 行异步复位优先. S 为 00 时保持, 不做操作; 01, 10 时移位; 11 时将输入送输出.

可以修改 01 的 L 为`(Algor ? D_out[ways - 1] : L)`来实现算术右移.

```
module Register #(
    parameter ways = 8
) (
    input [ways - 1 : 0] D_in,
    output reg [ways - 1 : 0] D_out,
    input CLK,
    input [1 : 0] S,
    input L, R,
    input RST
);
    always @(posedge CLK or posedge RST) begin
        if (RST) begin
            D_out <= 0;
        end
        else begin
            case (S)
```

```
                2'b00: ;
                2'b01: D_out <= {L, D_out[ways - 1 : 1]};
                2'b10: D_out <= {D_out[ways - 2 : 0], R};
                2'b11: D_out <= D_in;
            endcase
        end
    end
endmodule
```

Listing 8: 移位寄存器

2.2.4 44 进制计数器

判断是否要归零, 进行C_out置 0 或C_out自增 1.
注意这里是预置位电路, 归零发生在下一周期, 所以 11 行判断条件应当是: C_out == 6'd43.

```
module Counter_44 (
    input clk,
    input rst,
    output reg [5 : 0] C_out
);
    always @(posedge rst or posedge clk) begin
        if (rst) begin
            C_out <= 6'd0;
        end
        else begin
            if (C_out == 6'd44) begin
                C_out <= 6'd0;
            end
            else begin
                C_out <= C_out + 1'b1;
            end
        end
    end
endmodule
```

Listing 9: 44 进制计数器

2.3 verilog 仿真

2.3.1 D-锁存器

```
`timescale 1ns / 1ps
module D_Latch_tb;
    reg clk;
    reg d;
    reg rst;
    reg en;
    wire q;
    wire nq;
    D_Latch D_Latch_sim (
        .CLK(clk),
        .D(d),
        .RST(rst),
        .EN(en),
        .Q(q),
        .NQ(nq)
    );
    always begin
        clk = ~clk;
        #10;
    end
    initial begin
        clk = 0;
        d = 0;
        en = 0;
        rst = 0;

        #10;
        d = 1;
        #20;
        en = 1;
        #20;
        en = 0;
        #20;
        d = 0;
        #20;
```

```
36          rst = 1;
37          #20;
38          rst = 0;
39          #20;
40          en = 1;
41          #20;
42          d = 1;
43          #20;
44          d = 0;
45          #20;
46          $finish;
47      end
48
49
50      initial begin
51          $dumpfile("D_Latch_wave.vcd");
52          $dumpvars(0, D_Latch_tb);
53      end
54  endmodule
```

Listing 10: D-锁存器仿真

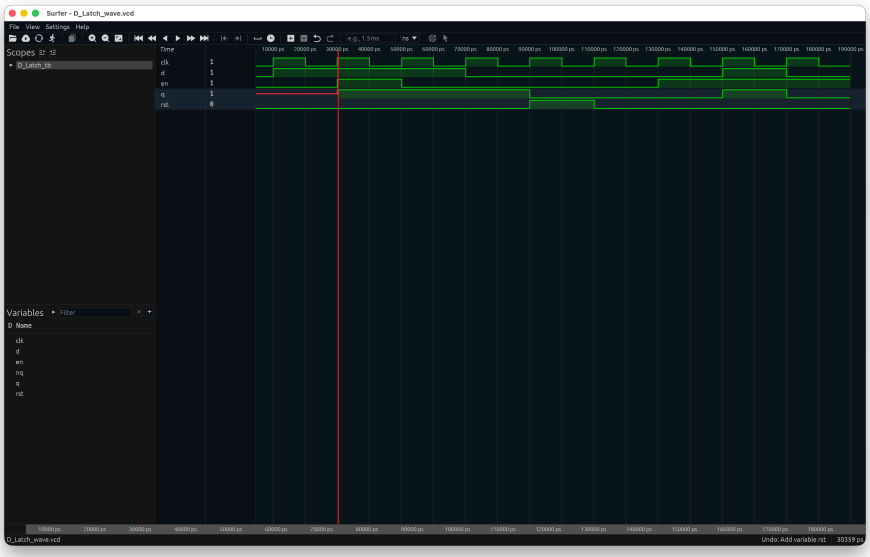


图 16: D-锁存器仿真波形

2.3.2 D-触发器

```
`timescale 1ns / 1ps
module D_Trigger_tb;
    reg clk;
    reg d;
    reg rst;
    reg en;
    wire q;
    wire nq;
    D_Trigger D_Trigger_sim (
        .CLK(clk),
        .D(d),
        .RST(rst),
        .EN(en),
        .Q(q),
        .NQ(nq)
    );
    always begin
        #12;
```

```
19          clk = ~clk;
20      end
21
22      initial begin
23          clk = 0;
24          d = 0;
25          rst = 0;
26          en = 0;
27          #15;
28          rst = 1;
29          #15;
30          rst = 0;
31          #500;
32          $finish;
33      end
34
35      always begin
36          d = ~d;
37          #20;
38      end
39
40      always begin
41          en = ~en;
42          #25;
43      end
44
45      initial begin
46          $dumpfile("D_Trigger_wave.vcd");
47          $dumpvars(0, D_Trigger_tb);
48      end
49  endmodule
```

Listing 11: D-触发器仿真

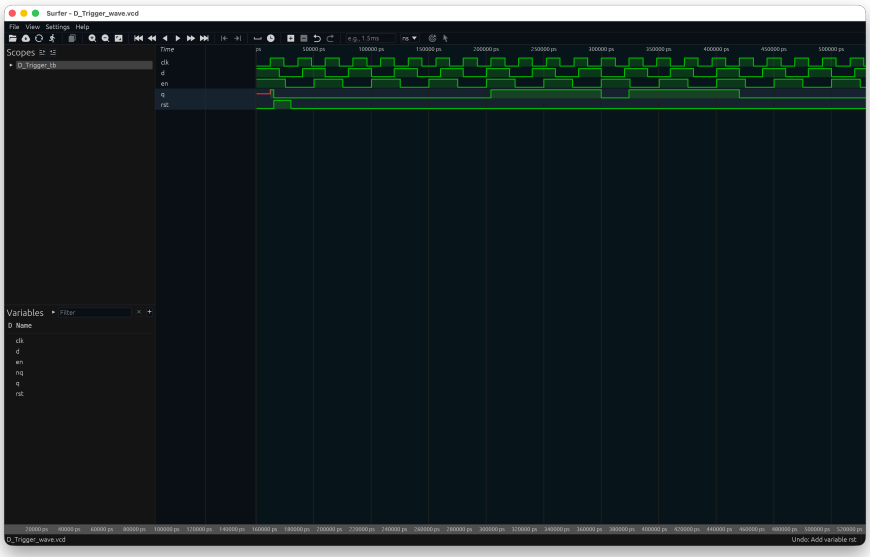


图 17: D-触发器仿真波形

2.3.3 移位寄存器

```
1  `timescale 1ns / 1ps
2  module Register_tb ();
3      parameter ways = 8;
4      reg [ways - 1 : 0] D_in;
5      wire [ways - 1 : 0] D_out;
6      reg clk;
7      reg [1 : 0] s;
8      reg l;
9      reg r;
10     reg rst;
11
12     Register #(
13         .ways(ways)
14     ) Register_sim (
15         .D_in(D_in),
16         .D_out(D_out),
17         .CLK(clk),
18         .S(s),
```

```
19          .R(r),
20          .L(l),
21          .RST(rst)
22      );
23
24      always begin
25          #10;
26          clk = ~clk;
27      end
28
29      initial begin
30          clk = 0;
31          D_in = 0;
32          s = 0;
33          r = 0;
34          l = 0;
35          rst = 0;
36          #10;
37
38          rst = 1;
39          #20;
40          rst = 0;
41          #10;
42          #800;
43          $finish;
44      end
45
46      integer i;
47      always begin
48          for (i = 0; i < (1 << ways); i = i + 1) begin
49              D_in = i;
50              #7;
51          end
52      end
53
54      always begin
55          s = 0;
56          #100;
57          s = 3;
```

```
58          #100;
59          s = 0;
60          #100;
61          s = 1;
62          #100;
63          s = 2;
64          #100;
65          s = 3;
66      end
67
68      initial begin
69          $dumpfile("Register_wave.vcd");
70          $dumpvars(0, Register_sim);
71      end
72  endmodule
```

Listing 12: 移位寄存器仿真

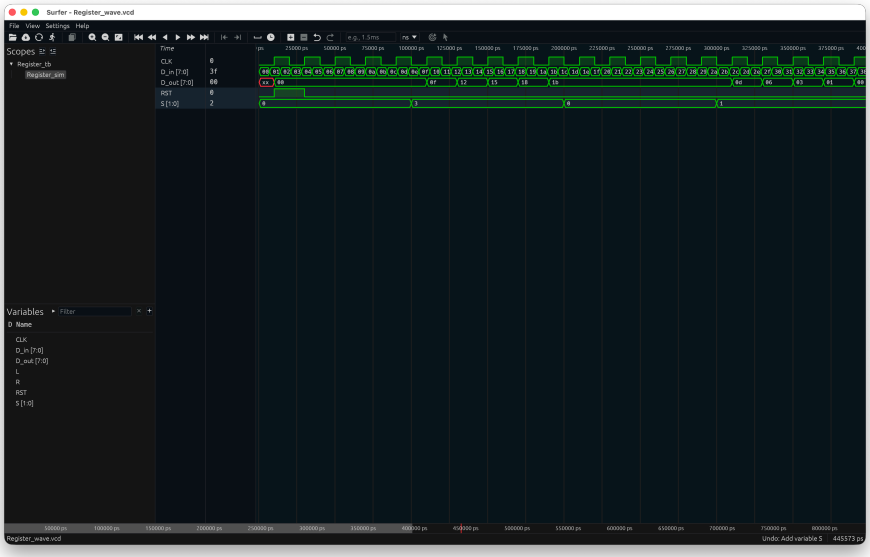


图 18: 移位寄存器仿真波形

2.3.4 44 进制计数器

```
`timescale 1ns / 1ps
module Counter_44_tb;
    reg clk;
    reg rst;
    wire [5 : 0] cnt;

    always begin
        #10;
        clk = ~clk;
    end

    Counter_44 Counter_44_sim (
        .clk(clk),
        .rst(rst),
        .C_out(cnt)
    );

    initial begin
        clk = 0;
        rst = 0;
        #10;
        rst = 1;
        #10;
        rst = 0;
        #1000;
        $finish;
    end

    initial begin
        $dumpfile("Counter_44_wave.vcd");
        $dumpvars(0, Counter_44_tb);
    end
endmodule
```

Listing 13: 44 进制计数器仿真

在仿真波形中可以看到, 计数不应当出现 0x2c, 这是 45 进制的计数器.

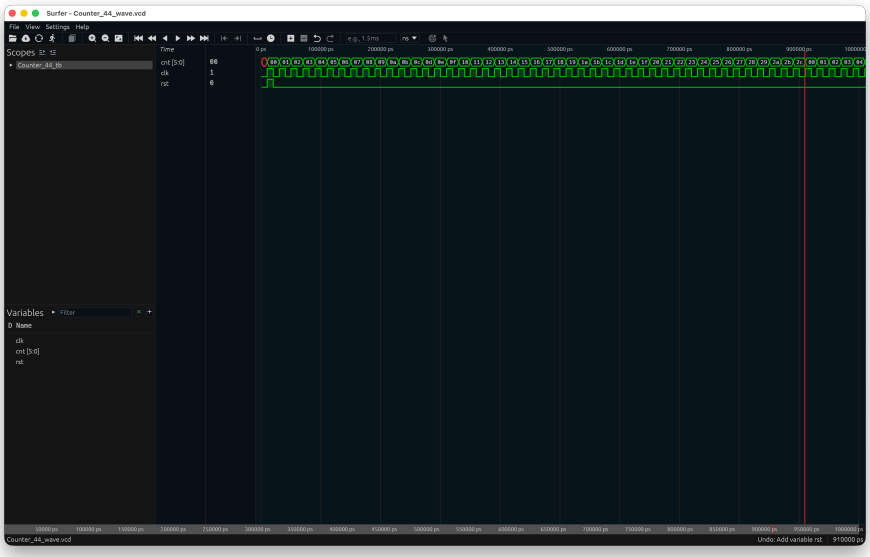


图 19: 44 进制计数器仿真波形

3 3